

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	DHANUSHREE S	Reg. No.:	192421218
Date:	17-08-2026	Duration:	60 minutes

Code of Conduct

I Dhanushree S/192421218 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Dhanushree S

Annexure A – Git & Docker Technical Assignment

AI-Based Crop Disease Detection and Farmer Advisory Platform

ABSTRACT

Agriculture plays a vital role in food production, and crop diseases can significantly reduce crop yield and quality. Early identification of plant diseases is important for helping farmers take suitable preventive actions. The AI-Based Crop Disease Detection and Farmer Advisory Platform is designed to detect crop diseases from uploaded plant leaf images using Artificial Intelligence and provide appropriate advisory information to farmers.

The system uses an image classification model to analyze crop leaf images and predict whether the plant is healthy or affected by a particular disease. Based on the prediction, the platform displays the detected disease, confidence score, symptoms, causes, and suggested preventive measures. The application can be developed using a Python-based backend framework such as FastAPI, an AI model using TensorFlow or PyTorch, and a database for storing prediction and advisory data.

This technical assignment applies Git for version control and Docker for containerization. Git helps manage source code, maintain commit history, support branching, and enable collaboration. Docker provides a consistent environment for running the AI application and its dependencies. Docker Compose is used to manage multiple services such as the application backend and database.

The proposed implementation improves portability, maintainability, reproducibility, and deployment efficiency. This report explains the complete Git and Docker workflow used for the project, including repository design, branching strategy, commits, Dockerfile development, Docker Compose configuration, container deployment, testing, challenges, and future improvements.

Keywords: Artificial Intelligence, Crop Disease Detection, Machine Learning, Deep Learning, Git, Docker, FastAPI, Containerization, Farmer Advisory.

1. PROBLEM ANALYSIS

1.1 Problem Statement

Farmers often identify crop diseases through visual inspection of leaves and plants. However, manual disease identification may be difficult because different diseases can have similar symptoms. Farmers may

also require agricultural experts to identify the disease correctly. Delays in disease detection can result in crop damage, reduced productivity, and economic loss.

The proposed AI-Based Crop Disease Detection and Farmer Advisory Platform solves this problem by using Artificial Intelligence to analyze images of crop leaves. A farmer uploads an image of an affected crop leaf to the system. The trained AI model processes the image and predicts the possible disease or determines whether the crop is healthy.

After detection, the platform provides useful advisory information, such as:

- Name of the detected disease
- Prediction confidence
- Disease symptoms
- Possible causes
- Preventive measures
- Basic treatment recommendations
- General crop care suggestions

The project combines AI-based image classification with a user-friendly application interface to provide faster agricultural support.

1.2 Project Objectives

The major objectives of the project are:

1. To develop an AI-based system for detecting crop diseases from leaf images.
2. To classify uploaded images as healthy or diseased.
3. To identify the specific disease category using a trained machine learning or deep learning model.
4. To display the prediction confidence score.
5. To provide suitable advisory and preventive recommendations to farmers.
6. To store prediction information for future analysis.
7. To maintain the project source code using Git.
8. To use Git branches for organized feature development.

9. To containerize the application using Docker.
10. To deploy multiple project services using Docker Compose.
11. To ensure the application runs consistently across different systems.

1.3 Functional Requirements

The system should perform the following functions:

FR1: Image Upload

The user should be able to upload a crop or leaf image through the application.

FR2: Image Validation

The system should validate the uploaded image format before processing.

Supported formats may include:

- JPG
- JPEG
- PNG

FR3: Image Preprocessing

The uploaded image should be resized and converted into the format required by the AI model.

FR4: Disease Prediction

The AI model should analyze the image and predict:

- Healthy crop, or
- Specific crop disease

FR5: Confidence Calculation

The system should display the confidence level of the prediction.

FR6: Farmer Advisory

The system should provide disease-related information and recommendations.

FR7: Prediction Storage

Prediction results may be stored in a database for monitoring and future analysis.

FR8: API Support

The backend should provide API endpoints for uploading images and retrieving predictions.

FR9: Containerized Deployment

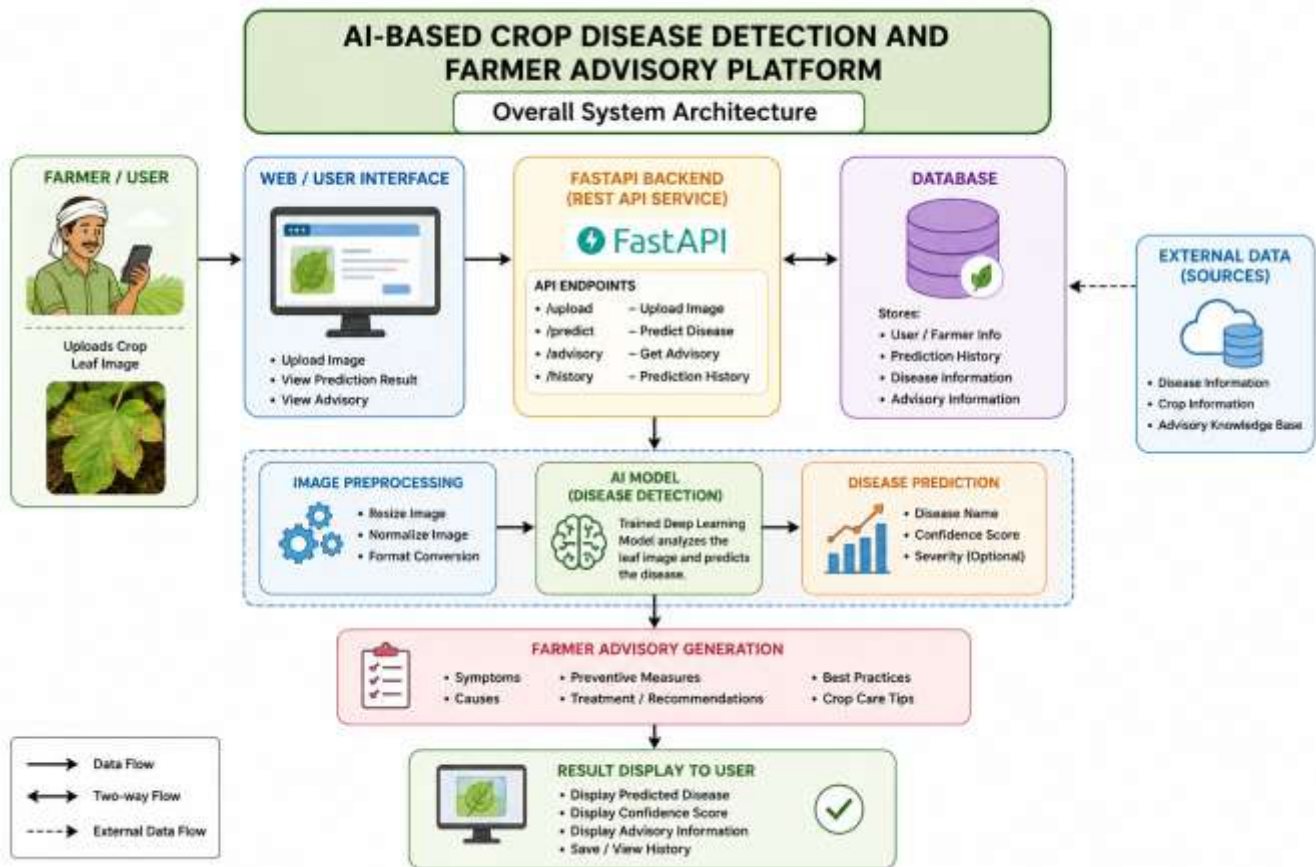
The complete application should run inside Docker containers.

1.4 Expected Outcomes

The expected outcomes of the project are:

- Successful image-based crop disease prediction.
- Faster identification of crop diseases.
- AI-generated or database-based farmer advisory.
- Proper source code version management using Git.
- Complete Git commit history.
- Separate branches for different development activities.
- Successful application containerization.
- Reproducible application environment.
- Easy deployment using Docker Compose.
- Reduced dependency and environment-related errors.

1.5 Overall System Architecture



2. PROJECT STRUCTURE DESIGN

A proper repository structure is important for organizing source code, AI models, configuration files, tests, and Docker files.

2.1 Proposed Folder Structure

AI-Crop-Disease-Detection/

|

|— app/

| |— __init__.py

```
| |— main.py
| |— config.py
| |
| |— api/
| | |— routes.py
| | |— schemas.py
| |
| |— services/
| | |— prediction_service.py
| | |— advisory_service.py
| |
| |— utils/
| | |— image_preprocessing.py
| | |— helpers.py
|
|— model/
| |— crop_disease_model.h5
|
|— data/
| |— sample_images/
```

```
|
|
├── tests/
|   ├── test_api.py
|   └── test_prediction.py
|
|
├── docs/
|   ├── architecture.png
|   └── workflow.png
|
|
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .dockerignore
├── .gitignore
├── README.md
└── .env.example
```

2.2 Purpose of Each Major Folder and File

Folder/File	Purpose
app/	Contains the main application source code

app/main.py	Entry point of the FastAPI application
app/api/	Contains API routes and request/response schemas
app/services/	Contains prediction and farmer advisory business logic
app/utils/	Contains reusable utility functions
model/	Stores the trained AI crop disease detection model
data/	Contains sample images or project datasets
tests/	Contains unit and API testing files
docs/	Contains diagrams and technical documentation
requirements.txt	Lists required Python libraries
Dockerfile	Defines instructions for building the application container
docker-compose.yml	Defines multiple Docker services
.gitignore	Specifies files Git should not track
.dockerignore	Specifies unnecessary files excluded from Docker build
README.md	Provides project documentation and usage instructions
.env.example	Provides sample environment variable configuration

2.3 Justification of the Repository Structure

The proposed structure separates different responsibilities within the project. The app folder contains application logic, while the model folder stores the trained AI model. Tests are maintained separately to improve software quality. Docker configuration files are kept in the root directory for easy deployment.

This modular structure makes the project easier to understand, maintain, test, and extend. For example, a developer can update the advisory system without modifying the AI model implementation.

3. GIT REPOSITORY INITIALIZATION

Git is a distributed version control system used to track changes in source code. It allows developers to maintain previous versions, create branches, merge features, and collaborate effectively.

3.1 Git Repository Initialization Steps

First, create the project directory:

```
mkdir AI-Crop-Disease-Detection
```

```
cd AI-Crop-Disease-Detection
```

Initialize the Git repository:

```
git init
```

Check the repository status:

```
git status
```

Configure Git username and email if required:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your-email@example.com"
```

Create the project files and add them to the staging area:

```
git add .
```

Create the first commit:

```
git commit -m "Initial project structure for crop disease detection platform"
```

Rename or set the primary branch as main:

```
git branch -M main
```

Connect the local repository with the remote repository:

```
git remote add origin <repository-url>
```

Push the project:

```
git push -u origin main
```

3.2 Git Initialization Workflow



Figure 2: Git Repository Initialization Workflow

3.3 Purpose of Essential Git Files

.git

The .git directory is automatically created after executing:

```
git init
```

It contains Git metadata, commit history, branch information, configuration, and object data.

```
.gitignore
```

The .gitignore file prevents unnecessary files from being added to the repository.

Example:

```
__pycache__/
```

```
*.pyc
```

```
.env
```

```
venv/
```

```
.venv/
```

```
*.log
```

```
uploads/
```

```
model/*.h5
```

Large model files, temporary files, virtual environments, secret environment variables, and cache files should generally not be committed unnecessarily.

```
README.md
```

This file describes:

- Project purpose
- Installation steps
- Technologies used

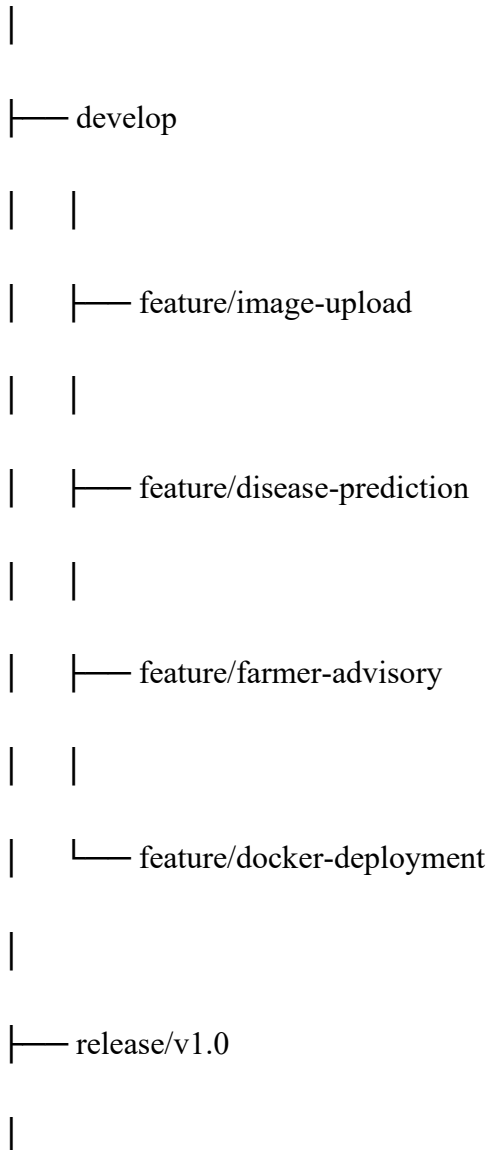
- Application usage
- API information
- Docker commands

4. GIT BRANCHING STRATEGY

A structured branching strategy is required to organize development activities.

The proposed project uses the following branches:

main



└── hotfix/critical-bug

4.1 Main Branch

The main branch contains stable and production-ready code.

Only tested and approved features should be merged into this branch.

4.2 Development Branch

The develop branch is used to integrate completed features before production release.

Developers merge tested feature branches into develop.

4.3 Feature Branches

Feature branches are created for specific project tasks.

Examples:

feature/image-upload

feature/disease-prediction

feature/farmer-advisory

feature/docker-deployment

Creation command:

git checkout develop

git checkout -b feature/disease-prediction

Or:

`git switch develop`

`git switch -c feature/disease-prediction`

4.4 Release Branch

A release branch is created when the project is ready for final testing.

Example:

`release/v1.0`

Command:

`git checkout -b release/v1.0 develop`

After testing, it can be merged into main.

4.5 Hotfix Branch

A hotfix branch is used to fix critical issues in the production version.

Example:

`hotfix/api-image-validation`

This approach ensures that urgent problems can be fixed without interrupting ongoing development.

4.6 Branching Strategy Justification

This branching model is suitable because the project contains multiple independent components:

- Image upload

- AI disease prediction
- Advisory generation
- Database integration
- API development
- Docker deployment

Each component can be developed independently in a feature branch. This reduces the risk of unstable code directly entering the production branch.

5. VERSION CONTROL WORKFLOW

The following Git workflow was used for project development.

5.1 Step 1: Check Current Status

`git status`

This command displays:

- Current branch
- Modified files
- Staged files
- Untracked files

5.2 Step 2: Add Files

`git add .`

This command adds all selected project changes to the staging area.

For a specific file:

`git add app/main.py`

5.3 Step 3: Create a Commit

```
git commit -m "feat: add crop disease prediction API"
```

A commit creates a snapshot of the current project changes.

5.4 Step 4: Create a Feature Branch

```
git checkout -b feature/farmer-advisory
```

This creates a separate workspace for implementing the farmer advisory functionality.

5.5 Step 5: Develop and Commit

Example:

```
git add app/services/advisory_service.py
```

```
git commit -m "feat: add disease advisory recommendation service"
```

5.6 Step 6: Merge the Feature

Switch to the development branch:

```
git checkout develop
```

Merge the completed feature:

```
git merge feature/farmer-advisory
```

5.7 Step 7: Synchronize With Remote Repository

Retrieve remote changes:

git pull origin develop

Push local changes:

git push origin develop

5.8 Example Git Workflow Diagram

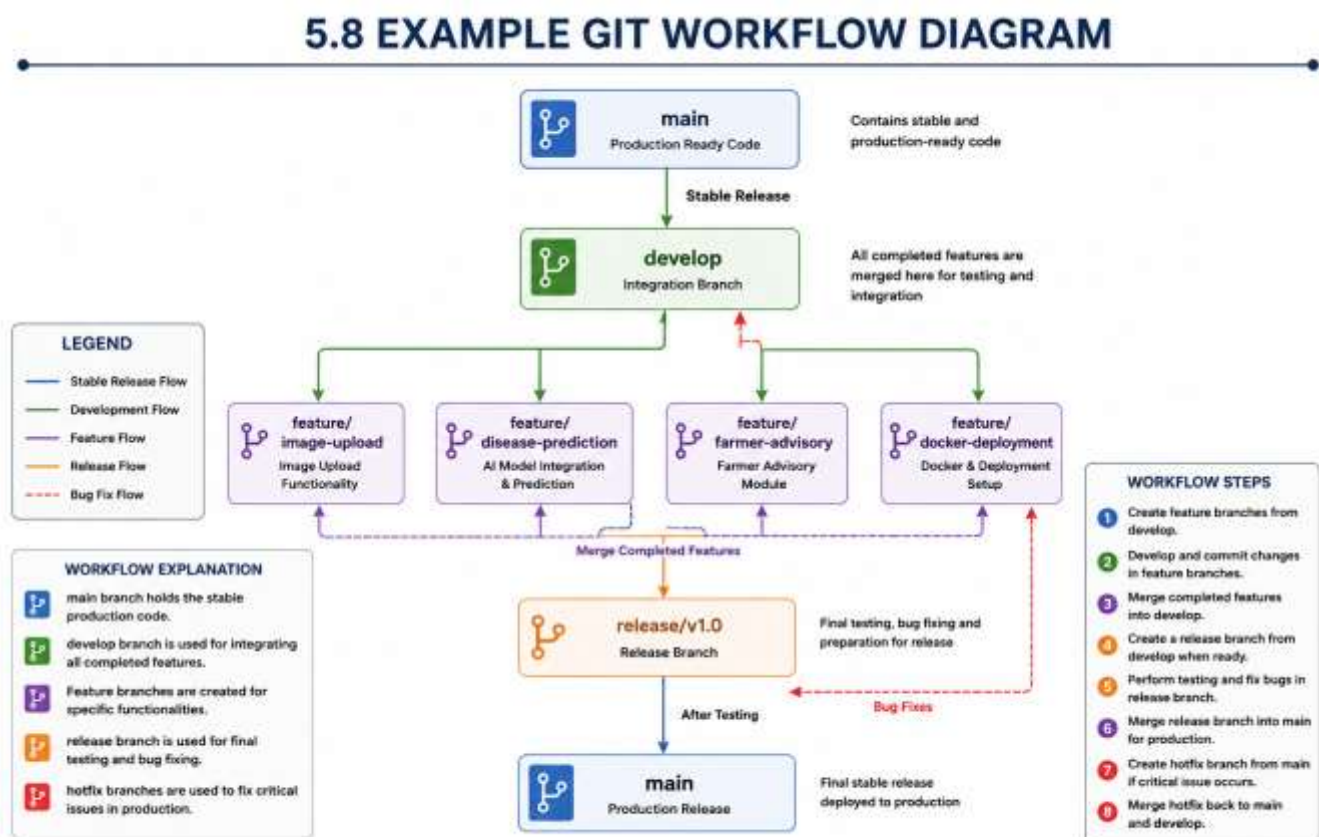


Figure 3: Git Branching and Development Workflow

5.9 Conflict Resolution

A merge conflict may occur when two branches modify the same section of a file.

Example conflict:

```
<<<<<<< HEAD
```

```
return {"message": "Prediction completed"}
```

```
=====
```

```
return {"message": "Disease detected successfully"}
```

```
>>>>>>> feature/disease-prediction
```

The developer manually selects or combines the required code.

After resolving the conflict:

```
git add app/main.py
```

```
git commit -m "fix: resolve merge conflict in prediction response"
```

Conflict resolution is important because Git cannot automatically determine which version of conflicting code should be retained.

6. COMMIT HISTORY ANALYSIS

A good project should contain meaningful commits rather than one large commit containing all changes.

6.1 Example Commit History

```
a8f92d1 fix: resolve API image validation error
```

```
c4e781a docs: update Docker deployment instructions
```

```
f9a352b chore: add Docker Compose configuration
```

```
e3b7a10 build: add Dockerfile for FastAPI application
```

d2c9f45 feat: add farmer advisory service

b7e3a91 feat: integrate crop disease prediction model

a1d7e88 feat: add image upload API endpoint

7c6b2f4 chore: add project dependencies

3e4f1c2 Initial project structure for crop disease detection platform

The commit history can be viewed using:

```
git log --oneline
```

For a graphical branch history:

```
git log --oneline --graph --all
```

6.2 Commit Message Practices

The project uses clear commit messages based on common prefixes.

Prefix	Purpose
feat:	Adds a new feature
fix:	Fixes a problem
docs:	Updates documentation
build:	Changes build or Docker configuration
test:	Adds or modifies tests
chore:	Performs maintenance tasks
refactor:	Improves code structure

7. DOCKER ENVIRONMENT DESIGN

Docker is a containerization platform that packages an application and its dependencies into a container.

For this project, Docker is suitable because an AI-based application may require several dependencies, such as:

- Python
- FastAPI
- Uvicorn
- TensorFlow or PyTorch
- NumPy
- Pillow
- Database libraries

Installing all dependencies manually on every computer may lead to compatibility problems. Docker solves this issue by creating a consistent execution environment.

7.1 Components Requiring Containerization

The following components can be containerized:

Application Container

Contains:

- FastAPI backend
- API routes
- Image processing
- Disease prediction logic
- Farmer advisory service

Database Container

Stores:

- Prediction history
- Disease information
- Advisory data
- User information, if required

Future AI Service Container

For a larger deployment, the AI model can be separated into an independent prediction service.

7.2 Docker Architecture

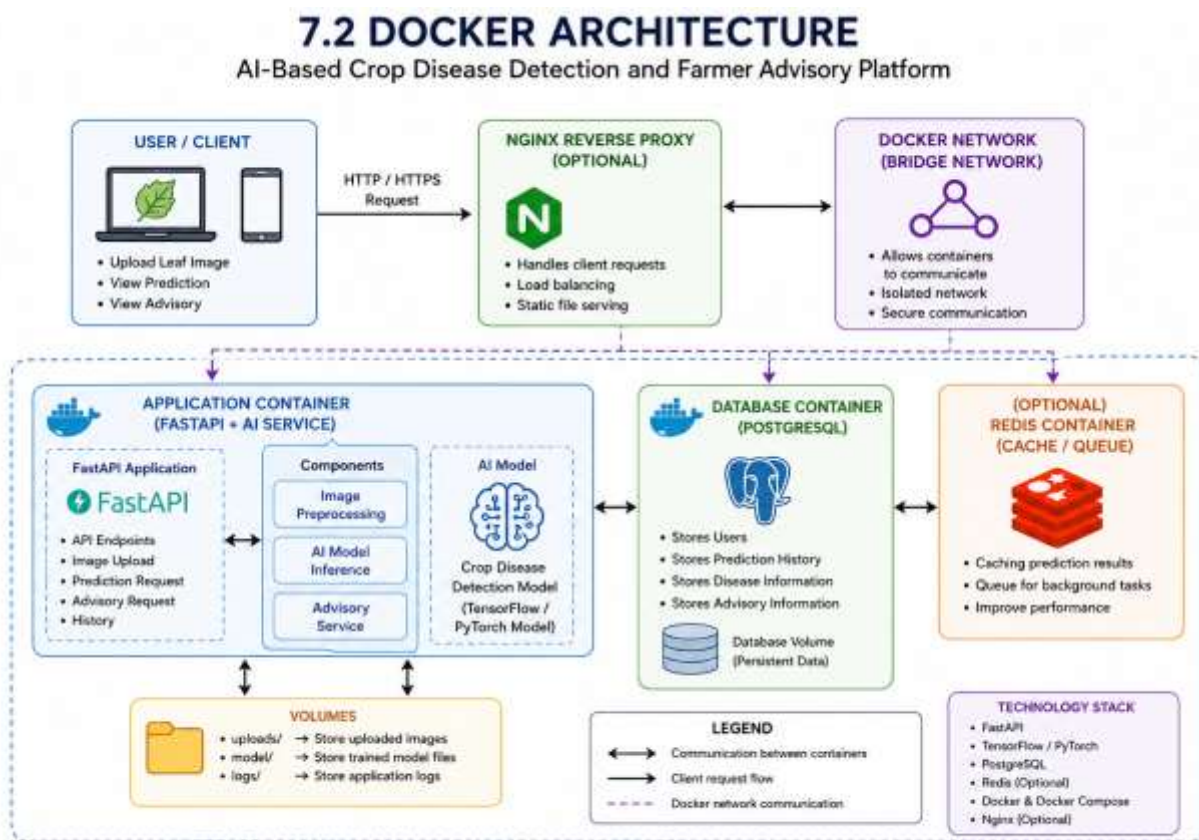


Figure: Docker Architecture of AI-Based Crop Disease Detection and Farmer Advisory Platform

Figure 4: Docker Container Architecture

8. DOCKERFILE DEVELOPMENT

The Dockerfile contains instructions for building the application image.

8.1 Dockerfile

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 8000
```

```
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

8.2 Explanation of Dockerfile Instructions

FROM

```
FROM python:3.11-slim
```

This specifies the base image containing Python 3.11.

WORKDIR

```
WORKDIR /app
```

This sets /app as the working directory inside the container.

COPY

COPY requirements.txt .

Copies the dependency file into the container.

RUN

RUN pip install --no-cache-dir -r requirements.txt

Installs the Python packages required by the project.

COPY Application Source Code

COPY . .

Copies the remaining project files into the container.

EXPOSE

EXPOSE 8000

Documents that the application uses port 8000.

CMD

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

Starts the FastAPI application when the container runs.

8.3 Docker Build Command

Build the Docker image:

```
docker build -t crop-disease-app .
```

Check available images:

```
docker images
```

Run the container:

```
docker run -p 8000:8000 crop-disease-app
```

The application can then be accessed at:

```
http://localhost:8000
```

For FastAPI API documentation:

```
http://localhost:8000/docs
```

9. DOCKER COMPOSE DESIGN

Docker Compose is used to define and run multiple containers using a single configuration file.

This project uses two major services:

1. app – AI-based crop disease detection application.
2. db – Database service.

9.1 docker-compose.yml

services:

app:

build: .

container_name: crop-disease-app

ports:

- "8000:8000"

environment:

DATABASE_URL: postgresql://cropuser:croppassword@db:5432/cropdb

MODEL_PATH: /app/model/crop_disease_model.h5

depends_on:

- db

volumes:

- ./uploads:/app/uploads

networks:

- crop-network

db:

image: postgres:16

container_name: crop-disease-db

restart: always

environment:

POSTGRES_DB: cropdb

POSTGRES_USER: cropuser

POSTGRES_PASSWORD: croppassword

volumes:

- postgres-data:/var/lib/postgresql/data

networks:

- crop-network

volumes:

postgres-data:

networks:

crop-network:

driver: bridge

9.2 Services Explanation

Application Service

The app service:

- Builds the Docker image using the Dockerfile.
- Runs the FastAPI application.
- Maps container port 8000 to host port 8000.
- Connects with the database.
- Uses environment variables.
- Stores uploaded files through a mounted volume.

Database Service

The db service:

- Uses PostgreSQL.
- Stores persistent project data.
- Uses a Docker volume.
- Connects to the application through the Docker network.

9.3 Networking

networks:

crop-network:

driver: bridge

The bridge network allows containers to communicate with each other.

The application can access the database using the service name:

db

instead of using an IP address.

9.4 Volumes

volumes:

postgres-data:

The database volume ensures that stored data is preserved even when the database container is restarted.

10. CONTAINER DEPLOYMENT AND TESTING

Start the Complete Application

Execute:

```
docker compose up --build
```

This command:

1. Builds the application image.
2. Creates the required network.
3. Creates the database volume.
4. Starts the database container.
5. Starts the application container.

For background execution:

```
docker compose up --build -d
```

Testing Table

Test ID	Test Case	Expected Result	Status
T1	Start Docker Compose	All services start successfully	Pass
T2	Access application root	API returns application message	Pass
T3	Open /docs	Swagger documentation opens	Pass
T4	Upload valid crop image	Image is accepted	Pass
T5	Upload invalid file	Validation error is returned	Pass
T6	Execute prediction	Disease prediction is returned	Pass
T7	Check confidence	Confidence value is displayed	Pass
T8	Verify database container	Database container runs successfully	Pass

Testing Workflow



Figure 5: Container Deployment and Testing Workflow

11. BENEFITS OF GIT AND DOCKER

11.1 Benefits of Git

Version Management

Git records every meaningful change in the project. Developers can compare versions and return to an earlier working version when required.

Collaboration

Multiple developers can work on different branches simultaneously.

For example:

Developer 1 → Image Upload

Developer 2 → AI Prediction

Developer 3 → Farmer Advisory

Developer 4 → Docker Deployment

Git allows these changes to be merged systematically.

Branching

Feature branches isolate unfinished development from stable code.

Commit History

The commit history documents the development process and makes it easier to identify when a change was introduced.

Backup and Synchronization

Remote repositories provide an additional location for storing and synchronizing project code.

11.2 Benefits of Docker

Portability

The application can run on different systems with the same Docker configuration.

Consistent Environment

All required dependencies are packaged into the container environment.

Simplified Deployment

The application can be started using:

`docker compose up`

Isolation

The project environment is isolated from the host operating system.

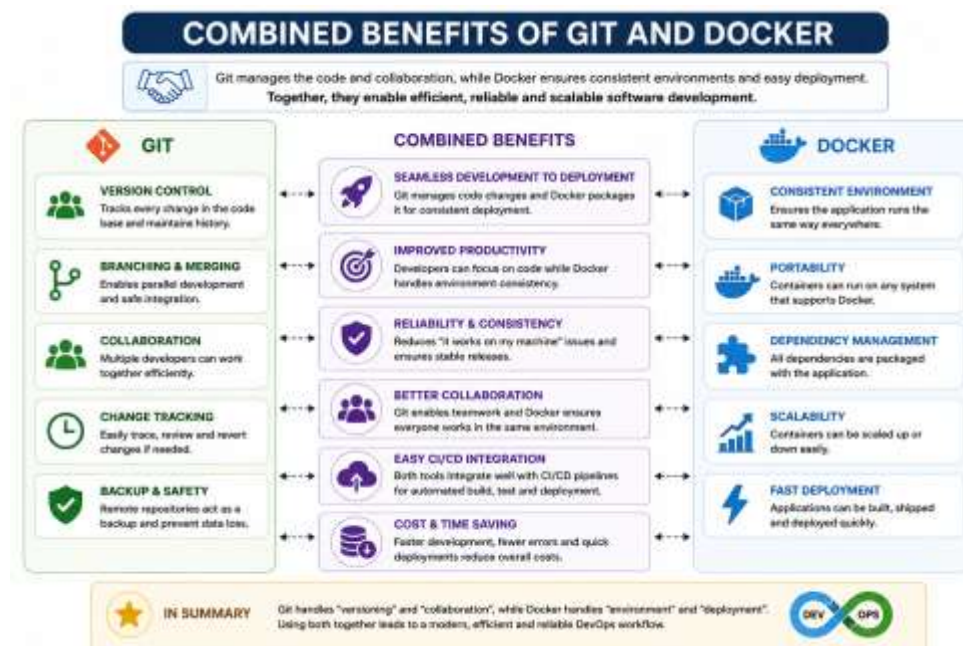
Scalability

Containers can be replicated or scaled based on future application requirements.

Maintainability

Application configuration and dependencies are clearly defined in Docker files.

11.3 Combined Benefits of Git and Docker



Git and Docker solve different but complementary problems.

CONCLUSION

The AI-Based Crop Disease Detection and Farmer Advisory Platform is designed to support farmers by using Artificial Intelligence for early crop disease identification. The system accepts crop leaf images, processes them using an AI model, predicts possible diseases, and provides advisory information.

The technical implementation of the project benefits significantly from the use of Git and Docker. Git provides an organized method for tracking changes, maintaining commit history, creating branches, merging features, resolving conflicts, and supporting collaboration. A structured branching strategy ensures that new features can be developed without directly affecting stable code.

Docker solves environment and deployment problems by packaging the application and its dependencies into containers. The Dockerfile defines how the AI application is built, while Docker Compose manages multiple services such as the FastAPI application and PostgreSQL database. Docker volumes provide persistent storage, and Docker networking enables communication between services.

The combined use of Git and Docker improves the overall quality of the project by providing better version control, portability, reproducibility, maintainability, and deployment consistency. In the future, the platform can be enhanced with improved AI models, more crop categories, multilingual farmer support, real-time notifications, cloud deployment, automated CI/CD pipelines, and scalable container orchestration.

Thus, Git and Docker provide a strong technical foundation for developing and deploying the AI-Based Crop Disease Detection and Farmer Advisory Platform efficiently.

REFERENCES

1. Chacon, S. and Straub, B., *Pro Git*, Git Documentation.
2. Docker Inc., *Docker Documentation*, Docker.
3. Docker Inc., *Docker Compose Documentation*, Docker.
4. FastAPI, *FastAPI Documentation*.
5. Python Software Foundation, *Python Documentation*.

6. PostgreSQL Global Development Group, *PostgreSQL Documentation*.
7. TensorFlow, *TensorFlow Documentation*.
8. Shorten, C. and Khoshgoftaar, T. M., “A survey on Image Data Augmentation for Deep Learning,” *Journal of Big Data*.
9. Recent research articles on deep learning-based plant disease detection and image classification may be added according to the AI model and dataset selected for the final project.

APPENDIX A – IMPORTANT GIT COMMANDS

Initialize repository

```
git init
```

Check repository status

```
git status
```

Add all files

```
git add .
```

Add specific file

```
git add app/main.py
```

Create commit

```
git commit -m "feat: add crop disease prediction"
```

View branches

```
git branch
```

Create and switch to branch

```
git checkout -b feature/disease-prediction
```

Switch branch

```
git checkout develop
```

Merge branch

```
git merge feature/disease-prediction
```

View commit history

```
git log --oneline
```

View graphical history

```
git log --oneline --graph --all
```

Add remote repository

```
git remote add origin <repository-url>
```

Push branch

```
git push -u origin main
```

Pull remote changes

```
git pull origin main
```

Clone repository

```
git clone <repository-url>
```

APPENDIX B – IMPORTANT DOCKER COMMANDS

Check Docker version

```
docker --version
```

Build Docker image

```
docker build -t crop-disease-app .
```

View Docker images

```
docker images
```

Run application container

```
docker run -p 8000:8000 crop-disease-app
```

View running containers

```
docker ps
```

View all containers

```
docker ps -a
```

Stop a container

```
docker stop crop-disease-app
```

Start Docker Compose services

```
docker compose up
```

Build and start services

```
docker compose up --build
```

Run services in background

docker compose up -d

Stop Compose services

docker compose down

View application logs

docker compose logs app

View database logs

docker compose logs db

List Docker volumes

docker volume ls

List Docker networks

docker network ls

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.

Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25